**EXAM INSTRUCTIONS**

1. This exam must be completed individually. No communication or collaboration with others is permitted.
2. This exam consists of **only one question**, with a total of **100 points**, accounting for **15% of the final course grade**.
3. **Academic honesty is essential. Any submission found to be identical, in whole or in part, to another student's work will receive a score of zero for this exam immediately, without any warning or further discussion.**
4. Make sure your answers are clear and complete. Show your reasoning where required.
5. Submit your work by **May 29, 2026, at 13:30**. Late submissions will not be accepted unless approved in advance.
6. If you have any problems during the exam, please contact the instructor immediately at **kharittha.jan@kmutt.ac.th** or via **Facebook: Poy J. Kharittha**.

**By submitting your work, you confirm that it is entirely your own.**

---

**Topic: Automatic Defect Detection and Counting in Colored Objects**

Based on

- Color Image Processing,
- Binary Image Processing,
- Image Segmentation,
- Image Features, and
- Classification and Object Detection

---

You are required to create, collect, or generate your own image dataset containing multiple colored objects under realistic imaging conditions. The dataset should include challenges such as

- uneven illumination,
- touching objects,
- shadows,
- noise,
- holes,
- cracks,
- overlapping objects,
- or cluttered backgrounds.

Your task is to DESIGN a computer vision pipeline that

1. segments the target objects,
2. converts the result into a reliable binary mask,
3. cleans the mask using morphology,
4. extracts object boundaries,
5. extracts meaningful image features,
6. detects defective objects, and
7. reports quantitative measurements.

Choose ONE application domain:

- fruits,
- pills/capsules,
- rice grains,
- candies,
- leaves,
- industrial parts,
- colored screws,
- cells,
- or similar colored objects.

**Rules**

*Allowed Libraries and Tools*

You MAY use the following:

- numpy
- opencv-python
- matplotlib
- scipy
- scikit-image
- scikit-learn
- MATLAB

*Not Allowed*

You may **NOT** use:

- Pre-trained deep learning models
- Object detection frameworks such as YOLO, Faster R-CNN, and Mask R-CNN
- Ready-made defect detection frameworks,
- Fully automatic built-in pipelines that directly solve the entire problem (e.g., a complete watershed segmentation pipeline without your own implementation and analysis)

*Important Requirement*

You are expected to implement the core image-processing pipeline yourself. Your solution should clearly demonstrate your own understanding and implementation of techniques such as

- preprocessing,
- segmentation,
- feature extraction,
- object separation,
- defect detection,
- and post-processing.

**Dataset Requirements**

You must prepare a dataset containing:

- **12–15 images**
- **At least 2 object conditions or categories** (Example: different colors, shapes, sizes, or defect conditions)
- **At least 5 defective objects** in total across the dataset

Your dataset must include **at least THREE** of the following challenges:

- Uneven illumination
- Touching objects
- Shadows
- Noise
- Blur
- Holes or Cracks
- Overlapping objects
- Cluttered background

You may use:

- Your own captured images
- Synthetic/generated images
- Publicly available datasets

The dataset should be challenging enough to require meaningful image-processing techniques rather than trivial thresholding only.

**Part 1 — Segmentation and Binary Mask (25%)**
You must perform the following:
1. **Compare at least TWO segmentation methods** from the list below:
   - Global thresholding
   - Adaptive thresholding
   - Edge-based segmentation
   - Color thresholding
   - K-means clustering
2. **Generate a binary mask** from the segmentaion result
3. **Apply morphological operations** to improve the binary mask (Example: erosion, dilation, opening, or closing)

*Required Outputs*
For each selected method, show the following results:
- Original image
- Image histogram
- Segmented result
- Binary mask
- Result after morphology
- Labeled or counted objects

*Required Analysis*
Discuss the following:
1. Why some segmentation methods fails under uneven lighting conditions
2. Why touching or overlapping objects are difficult to separate
3. How morphological operations improve the segmentaion result
4. Failure cases and limitaions of your approach (Example: missed objects, incorrect segmentation, merged objects, noise sensitivity, etc.)

---

**Part 2 — Feature Extraction (25%)**
Extract features from each detected object.

*Required Features*
You must use:
1. **At least ONE shape feature**

Choose at least one from the following:
- Area
- Perimeter
- Circularity
- Aspect ratio
- Solidity

**2. At least ONE color or texture feature**

Choose at least one from the following:
- Mean RGB or HSV values
- Color histogram
- Intensity statistics

3. **At least ONE local feature**

Choose at least one from the following:
- Harris corners
- SIFT
- ORB

*Required Outputs*
Show the following results:

- Object contours
- Detected keypoints or corners
- Feature visualization
- Comparison between normal and defective objects

*Required Analysis*
Discuss the following:

1. Which features are most effective for detecting defects, and why
2. Which features are sensitive to illumination changes or shadows
3. Why cracks or holes may generate additional corners or keypoints
4. Why contour-based features may fail when objects are touching or overlapping

---

**Part 3 — Defect Detection and Counting (25%)**
Classify each object as either normal or defective.
*Allowed Methods*
You may use:

- Rule-based classification
- SVM
- Random Forest
- k-NN

*Required Outputs*
Show the following results:

- Bounding boxes around objects
- Object labels (normal/defective)
- Detected defects
- Confusion matrix
- Final counting results

*Required Quantitative Measurements*
Report the following measurements:

- Total number of objects
- Number of defective objects
- Average object size
- Defect percentage

*Example*
Total Objects: 42
Defective Objects: 6
Average Area: 521 pixels
Defect Percentage: 14.29%

*Required Analysis*
Discuss the following:

- The limitation of small-sample evaluation and how results might differ with larger datasets

**Scoring Rubric for PDF Report**: Pipeline and Answers for Part 1 to Part 3 (25%)

1. Pipeline Diagram/Flowchart — 5%
   Clear workflow showing the main processing steps and data flow of the system
2. Explanation of Parameter Choices — 5%
   Clear explanation and justification of important parameter settings and their effects on the results
3. Analysis and Discussion Quality — 10%
   Meaningful analysis of results, including strengths, weaknesses, comparisons, and effects of different image conditions
4. Limitation and Failure Case Discussion — 5%
   Discussion of system limitations, failure cases, possible causes, and suggested improvements

**Important Notes**

High scores will be awarded for:

- Clear reasoning and explanation
- Justification of parameter choices
- Comparison between different methods
- Analysis of limitaions and failure cases

You will NOT receive full scores for:

- Simply calling OpenCV functions without explanation
- Using default parameters without justification
- Showing outputs without analysis or discussion

*The implementation must strictly follow the pipeline described in PDF file. Any mismatch between explanation and submitted code will result in score deduction.*

---

**Submission Instructions (Upload to LEB2)**

You must submit:

1. One PDF file named: '*studentID_M2.pdf*'. The PDF (**25%**) must include:
   - Pipeline design and explanation
   - Answers for Part 1 to Part 3
2. One source code file named in one of the following formats:
   - '*S_studentID_M2.py*', or
   - '*S_studentID_M2.ipynb'*, or
   - *'S_studentID_M2.m'*

   (Python or MATLAB only**)**
3. Input images (You may submit them as a .zip folder)
4. Output images (You may submit them as a .zip folder)
5. README explaining how to run the code

Please ensure that:

- The filenames strictly follow the specified format.
- The submitted code matches the pipeline described in the PDF.
- All required files are included before submission.